

TALLER DE PROGRAMACION



Cómo vamos a Trabajar

Planteo de un tema

→ Conceptos teóricos

Ejercicios prácticos

Conclusiones

Formulario de preguntas

Conceptos de la clase de hoy

1

Estructuras
conocidas

2

Ordenación
de Datos

3

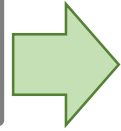
Método N-
pasadas

4

Conclusiones

Estructuras conocidas- ARREGLOS

ARREGLO



Un arreglo (ARRAY) es una estructura de datos compuesta que permite acceder a cada componente por una variable índice, que da la posición de la componente dentro de la estructura de datos.

HOMOGENEA: los elementos son del mismo tipo.

ESTATICA: la cantidad máxima de elementos se definen al momento de declarar la estructura y por lo tanto su tamaño máximo es fijo durante el programa.

LINEAL: cada elemento tiene un anterior y un siguiente (salvo el primero y el último).

INDEXADA: a los elementos se accede por medio de un índice.

Estructuras conocidas- ARREGLOS

Program ejemplo;

Const

... •

Type

nombre = **array** [LI..LS] of tipoElemento;

var

variable: nombre;

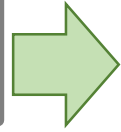
SIEMPRE manejan una dimensión física.

PUEDEN manejar una dimensión lógica.

OPERACIONES: Cargar la estructura, Agregar un elemento, Insertar un elemento, Eliminar un elemento, Recorrer la estructura, Buscar un elemento.

Estructuras conocidas- LISTAS

LISTA



Es una colección de nodos. Cada nodo contiene un elemento (valor que se quiere almacenar en la lista) y una dirección de memoria dinámica que indica donde se encuentra el siguiente nodo de la lista. Toda lista debe tener un nodo inicial.

HOMOGENEA: los elementos son del mismo tipo.

DINAMICA: la cantidad de elementos **puede variar** durante la ejecución del programa

LINEAL: cada elemento tiene un anterior y un siguiente (salvo el primero y el último).

SECUENCIAL: para acceder al elemento n -ésimo de la lista debo pasar por los $(n-1)$ anteriores.

Estructuras conocidas- LISTAS

Type

```
lista= ^nodo;
```

```
nodo = record  
    dato:tipo;  
    sig: lista;  
end;
```

SIEMPRE manejan una dirección inicial.

PUEDEN agregar o eliminar datos.

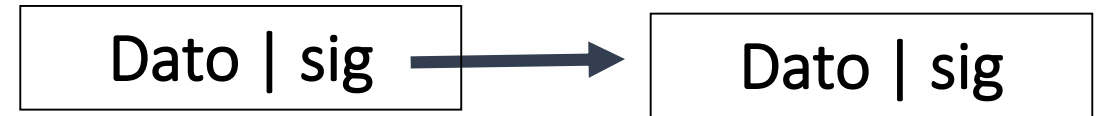
OPERACIONES: Crear una lista vacía, Agregar un elemento adelante, Agregar un elemento atrás, Insertar un elemento, Eliminar un elemento, Recorrer la estructura, Buscar un elemento

Estructuras conocidas- ARREGLOS vs LISTAS

➡ Cuál utilizamos?



Cuál es más eficiente?



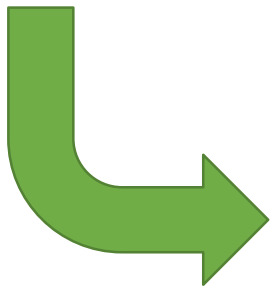
Pueden ordenarse?

Ordenacion- Objetivo

Ordenación



Es un proceso por el cual un conjunto de elementos puede ser ordenado.



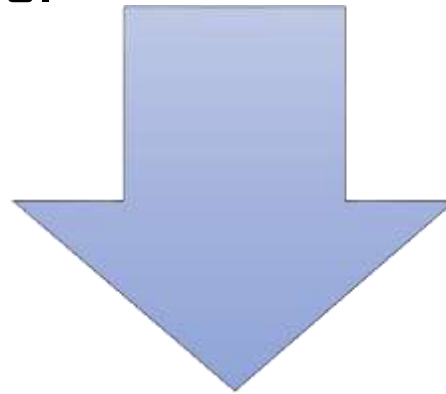
Ordenacion- Objetivo

Ordenación



Es un proceso por el cual un conjunto de elementos puede ser ordenado.

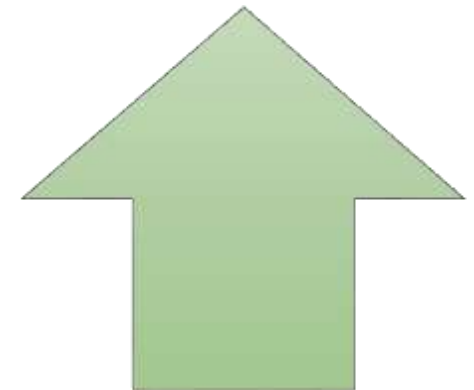
SELECCION
INTERCAMBIO
INSERCIÓN
QUICKSORT



Memoria
Tiempo



Facilidad de
implementación



Ordenación- SELECCION

Dado un arreglo A y una dimensión lógica ($dimL$), el algoritmo repite ($dimL-1$) veces buscar el elemento mínimo y ubicarlo en la posición correspondiente.

Es decir, en la primera vuelta busca el mínimo desde la posición 1 hasta la dimensión lógica y ubica al mínimo en la primera posición. En la vuelta 2 busca el mínimo desde la posición 2 hasta la dimensión lógica y lo ubica en la posición 2.

En la vuelta i busca el mínimo desde la posición i hasta la dimensión lógica y lo ubica en la posición i .

Ordenación- SELECCION- Pasos



DimF 7
DimL 5

VUELTA 1

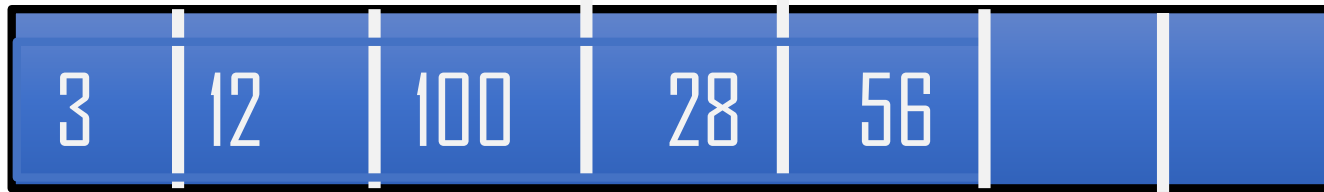
Busco en que posición
está el mínimo (1..dimL)

Pos 4

Intercambio $v[1]$ con $v[4]$



Ordenación- SELECCION- Pasos



DimF 7
DimL 5

VUELTA 2

Busco en que posición
está el mínimo (2..dimL)

Pos 2

Intercambio $v[2]$ con $v[2]$



Ordenación- SELECCION- Pasos



DimF 7

DimL 5

VUELTA 3

Busco en que posición
está el mínimo (3..dimL)

Pos 4

Intercambio $v[3]$ con $v[4]$



Ordenación- SELECCION- Pasos



DimF 7
DimL 5

VUELTA 4

Busco en que posición
está el mínimo (4..dimL)

Pos 5

Cómo se
implementa?

Intercambio $v[4]$ con $v[5]$



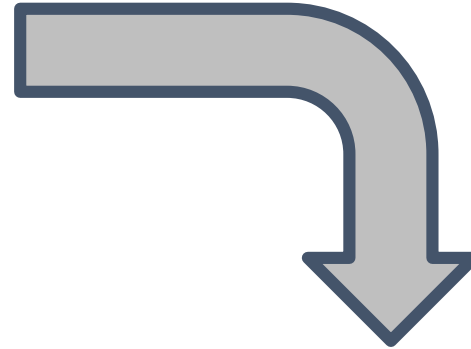
Ordenación- SELECCION- Implementación

Program seleccion;

Const dimF = {*dimension física*}

Type

Vector = Array [1..dimF] of TipoElem;



Program seleccion;

Const dimF = 7

Type

vector = Array [1..dimF] of integer;

Ordenación- SELECCION- Implementación

Program seleccion;

Const dimF = 7

Type

vector = **Array** [1..dimF] **of** integer;

Var

v:vector;

dimL:integer;

Begin

cargar (v,dimL);

ordenar (v,dimL);

End.

Ordenación- SELECCION- Implementación

```
Procedure Ordenar ( var v: vector; dimL: integer);
```

```
var
```

```
    i, j, pos, item: integer;
```

```
begin
```

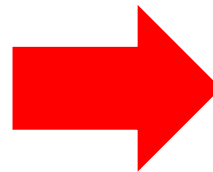
```
    for i:= 1 to (dimL-1) do
```

```
        begin
```

```
            pos := i;
```

```
            for j := (i+1) to dimL do
```

```
                if v[j] < v[pos] then pos:=j;
```

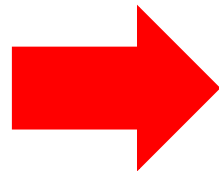


Guarda en pos la
posición donde está el
elemento mas chico

```
            item := v[pos];
```

```
            v[pos] := v[i];
```

```
            v[i] := item;
```



Realiza el
intercambio

```
        end;
```

```
    end;
```

Ordenación- CONCLUSIONES



Es muy sencilla la implementación.

Requiere un tiempo de ejecución del orden de N^2 , donde N es la dimensión lógica del arreglo.

Importa el orden original de los datos?

Formulario de preguntas